

Graph Neural Network

Hyoungh Joon, Kim

June 3, 2026

Contents

1	Introduction	2
2	Main Ideas	2
2.1	Basic Definitions	2
2.2	Permutation Invariance	2
2.3	Neural Message Passing	2
2.3.1	Graph Convolutional Network	3
2.3.2	Aggregate Operator	3
2.3.3	Update Operator	3
2.4	Node Classification	4
2.5	Edge Classification	4
2.6	Graph Classification	4
3	Extensions of Graph Neural Networks	4
3.1	Graph Attention Networks	4
3.2	Edge and Graph Embeddings	5
3.3	Geometric Deep Learning	5

1 Introduction

Graph neural networks are used to capture the information under the graph structure. Its main usage are:

- Graph classification: Classify the graph itself; ex. Molecule property prediction;
- Link prediction: Predict if edge exists between nodes; ex. Recommender system;
- Node / Edge classification: Classify the nodes or edges of the graph; ex. FDS

2 Main Ideas

2.1 Basic Definitions

Definition 2.1.1 (Graph)

The **graph** $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ consists of set of **nodes** (or vertices) denoted by $\mathcal{V}$, and a set of **edges** (or links) denoted by $\mathcal{E}$. Each nodes are indexed with $n = 1, 2, \dots, N$ where N is the total number of nodes. Edges from n to m is denoted by (n, m) . If two nodes are linked by an edge, they are called **neighbors**, and set of all neighbors of node n is denoted by $\mathcal{N}(n)$.

We use **adjacency matrix** to denote the connections between nodes. If the edge exists between node n and m , then the element of adjacency matrix $A_{nm} = 1$, else $A_{nm} = 0$. If the graph is undirected, the adjacency matrix is symmetric.

2.2 Permutation Invariance

The graph structure is invariant under the permutation operation. That is, even if we change the numbering of the graph, the network prediction should not change. Or, the network prediction may change, applying the same permutation to the prediction without permutation. That is, the network prediction is equivariant under permutation. Precise definition is given below;

Definition 2.2.1 (Permutation Invariance and Equivariance)

Let $\mathbf{P}$ be a permutation matrix. Let $\mathbf{y}(\mathbf{X}, \mathbf{A})$ be a network prediction defined on the node data matrix $\mathbf{X}$ and adjacency matrix $\mathbf{A}$. If the network satisfies following condition, the network is **invariant** under permutation;

$$\mathbf{y}(\tilde{\mathbf{X}}, \tilde{\mathbf{A}}) = \mathbf{y}(\mathbf{X}, \mathbf{A})$$

where $\tilde{\mathbf{X}} = \mathbf{P}\mathbf{X}$ and $\tilde{\mathbf{A}} = \mathbf{P}\mathbf{A}\mathbf{P}^\top$. If the network satisfies following condition, the network is **equivariant** under permutation;

$$\mathbf{y}(\tilde{\mathbf{X}}, \tilde{\mathbf{A}}) = \mathbf{P}\mathbf{y}(\mathbf{X}, \mathbf{A}).$$

2.3 Neural Message Passing

To use the permutation invariance and equivariance as the inductive bias of our network, it is not a good idea to use ordinary operations or flattening the matrix. Instead we adopt the method of CNN, such as parameter sharing.

2.3.1 Graph Convolutional Network

The graph network layer should satisfy following requirements:

- Handle variable-length inputs: Since the number of nodes and edges are different along graphs;
- Must be differentiable;
- Scale well: Since graphs can be very large, like social network;

We use the embedding vector of nodes, $\mathbf{h}_n$, and use the neighbors of each nodes the collect the information and make pre-activations while satisfying invariance.

Algorithm 1: Simple message passing neural network

Input : Undirected graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$
Initial node embeddings $\{\mathbf{h}_n^{(0)}\}$
Aggregate($\cdot$) function
Update($\cdot, \cdot$) function

Output: Final node embeddings $\{\mathbf{h}_n^{(L)}\}$

```

1 for  $l \in \{0, \dots, L-1\}$  do
2    $\mathbf{z}_n^{(0)} \leftarrow \text{Aggregate}\left(\left\{\mathbf{h}_m^{(l)} : m \in \mathcal{N}(n)\right\}\right)$ 
3    $\mathbf{h}_n^{(l+1)} \leftarrow \text{Update}\left(\mathbf{h}_n^{(l)}, \mathbf{z}_n^{(l)}\right)$ 
4 return  $\{\mathbf{h}_n^{(L)}\}$ 

```

The Aggregate($\cdot$) function collects the information of neighbor nodes and make a vector $\mathbf{z}$. It must be invariant under the ordering of the embedding vector. The Update($\cdot, \cdot$) function then update the given node with the vector $\mathbf{z}$ and make a new node embedding vector.

2.3.2 Aggregate Operator

The possible forms for the Aggregate function are:

- Simple sum: $\text{Aggregate}\left(\left\{\mathbf{h}_m^{(l)} : m \in \mathcal{N}(n)\right\}\right) = \sum_{m \in \mathcal{N}(n)} \mathbf{h}_m^{(l)}$
- Divided sum: $\text{Aggregate}\left(\left\{\mathbf{h}_m^{(l)} : m \in \mathcal{N}(n)\right\}\right) = \frac{1}{|\mathcal{N}(n)|} \sum_{m \in \mathcal{N}(n)} \mathbf{h}_m^{(l)}$
or $\text{Aggregate}\left(\left\{\mathbf{h}_m^{(l)} : m \in \mathcal{N}(n)\right\}\right) = \frac{1}{\sqrt{|\mathcal{N}(n)||\mathcal{M}(m)|}} \sum_{m \in \mathcal{N}(n)} \mathbf{h}_m^{(l)}$
- Fully connected layer: $\text{Aggregate}\left(\left\{\mathbf{h}_m^{(l)} : m \in \mathcal{N}(n)\right\}\right) = \text{MLP}_{\theta}\left(\sum_{m \in \mathcal{N}(n)} \text{MLP}_{\phi}(\mathbf{h}_m^{(l)})\right)$ with learnable parameter θ and ϕ .

2.3.3 Update Operator

The possible forms for the Update function are:

- Update $(\mathbf{h}_n^{(l)}, \mathbf{z}_n^{(l)}) = f(\mathbf{W}_{self} \mathbf{h}_n^{(l)} + \mathbf{W}_{neigh} \mathbf{z}_n^{(l)} + \mathbf{b})$, where f is a nonlinear activation function and $\mathbf{W}_{self}$, $\mathbf{W}_{neigh}$ and $\mathbf{b}$ are the learnable weights and biases.

2.4 Node Classification

We can classify the nodes by applying softmax function at the last layer of the graph neural network and define a loss function as the cross-entropy loss across all nodes and all classes. We distinguish three types of nodes for training:

1. $\mathcal{V}_{train}$: Labeled, included in the message-passing operations and used to compute the loss function for training;
2. $\mathcal{V}_{trans}$: Unlabeled, included in the message-passing operations but not used to compute the loss function for training; Graph without transductive nodes can be considered as supervised learning.
3. $\mathcal{V}_{induct}$: Unlabeled, and does not participate in the training session, but participate in message-passing during inference session and labels are predicted; Graph without inductive nodes can be considered as semi-supervised learning.

2.5 Edge Classification

We can classify the edges by defining additional edge properties. For example, to predict if the edge exists or not, we can define the probability $p(n, m)$ for the presence of an edge between node n and m as $p(n, m) = \sigma(\mathbf{h}_n^\top \mathbf{h}_m)$.

2.6 Graph Classification

We can classify the graph itself, by training with a set of labeled graphs $\{\mathcal{G}_1, \dots, \mathcal{G}_N\}$. We may use the final node embeddings to yield predictions about graphs. The simplest form of such approach is to take the sum of final node embeddings and pass through some activation function;

$$\mathbf{y} = \mathbf{f} \left(\sum_{n \in \mathcal{V}} \mathbf{h}_n^{(L)} \right)$$

3 Extensions of Graph Neural Networks

3.1 Graph Attention Networks

We can apply attention mechanism to graph networks, too! We may define the Aggregate function as following:

$$\mathbf{z}_n^{(l)} = \text{Aggregate} \left(\left\{ \mathbf{h}_m^{(l)} : m \in \mathcal{N}(n) \right\} \right) = \sum_{m \in \mathcal{N}(n)} A_{nm} \mathbf{h}_m^{(l)}$$

where A_{nm} are the attention weights. The attention weights may be defined as

- $A_{nm} = \frac{\exp(\mathbf{h}_n^\top \mathbf{W} \mathbf{h}_m)}{\sum_{k \in \mathcal{N}(n)} \exp(\mathbf{h}_n^\top \mathbf{W} \mathbf{h}_k)}$ where $\mathbf{W}$ is a learnable parameter.
- $A_{nm} = \frac{\exp(\text{MLP}(\mathbf{h}_n, \mathbf{h}_m))}{\sum_{k \in \mathcal{N}(n)} \exp(\text{MLP}(\mathbf{h}_n, \mathbf{h}_k))}$ where MLP has a single continuous output and invariant to the ordering of the node. If the MLP is shared across all nodes in graph, this aggregation function is equivariant under node reordering.

3.2 Edge and Graph Embeddings

We can introduce edge embedding vectors to manage the graph whose data is associated with edges. We can also consider the graph embeddings, which relates to the whole graph. This additional embeddings allow us to make richer set of graph features, and a general neural message-passing algorithm.

Algorithm 2: General message passing neural network

Input : Undirected graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$
Initial node embeddings $\{\mathbf{h}_n^{(0)}\}$, Initial edge embeddings $\{\mathbf{e}_{nm}^{(0)}\}$, Initial graph embedding $\{\mathbf{g}^{(0)}\}$
Aggregate_{node}($\cdot$) function
Update_{node}($\cdot, \cdot, \cdot$) function, Update_{edge}($\cdot, \cdot, \cdot, \cdot$) function, Update_{graph}($\cdot, \cdot, \cdot$) function

Output: Final node embeddings $\{\mathbf{h}_n^{(L)}\}$
Final edge embeddings $\{\mathbf{e}_{nm}^{(L)}\}$
Final graph embedding $\{\mathbf{g}^{(L)}\}$

```

1 for  $l \in \{0, \dots, L-1\}$  do
2    $\mathbf{e}_{nm}^{(l+1)} \leftarrow \text{Update}_{\text{edge}}(\mathbf{e}_{nm}^{(l)}, \mathbf{h}_n^{(l)}, \mathbf{h}_m^{(l)}, \mathbf{g}^{(l)})$ 
3    $\mathbf{z}_n^{(l+1)} \leftarrow \text{Aggregate}_{\text{node}}\left(\left\{\mathbf{e}_{nm}^{(l+1)} : m \in \mathcal{N}(n)\right\}\right)$ 
4    $\mathbf{h}_n^{(l+1)} \leftarrow \text{Update}_{\text{node}}\left(\mathbf{h}_n^{(l)}, \mathbf{z}_n^{(l+1)}, \mathbf{g}^{(l)}\right)$ 
5    $\mathbf{g}^{(l+1)} \leftarrow \text{Update}_{\text{graph}}\left(\mathbf{g}^{(l)}, \{\mathbf{h}_n^{(l+1)}\}, \{\mathbf{e}_{nm}^{(l+1)}\}\right)$ 
6 return  $\{\mathbf{h}_n^{(L)}\}, \{\mathbf{e}_{nm}^{(L)}\}, \mathbf{g}^{(L)}$ 

```

3.3 Geometric Deep Learning

Consider the task of predicting the properties of a molecule. The molecule can be represented as a list of spatial coordinates and types. If we use neural network for prediction, the network must be invariant of reordering of the nodes, and specific coordinate system. To meet these requirements, we can use graph neural network with embedding vector $\mathbf{r}_n$, with a slightly different Aggregate function and Update function; we replace the place of node embeddings with relative distance between nodes such as squared distances. For more information of these functions, see Satorras, Hoogeboom, and Welling, 2021 or Bishop, 2023.

Capturing such geometric symmetric features and taking it into account of the structure of the network forms the basis of a rich field of **geometric deep learning** (Bronstein *et al.*, 2017; Bronstien *et al.*, 2021).